Penalized regression

David Puelz
The University of Austin

Where we left off

- The Texas counties again: predict **physicians per 10,000 people** from 10 county features.
- Last time, stepwise selection and the full model tied under cross-validation.
- With 168 counties and only 10 predictors, there was little room to overfit.

Today's question: what happens when we give the model a lot more freedom?

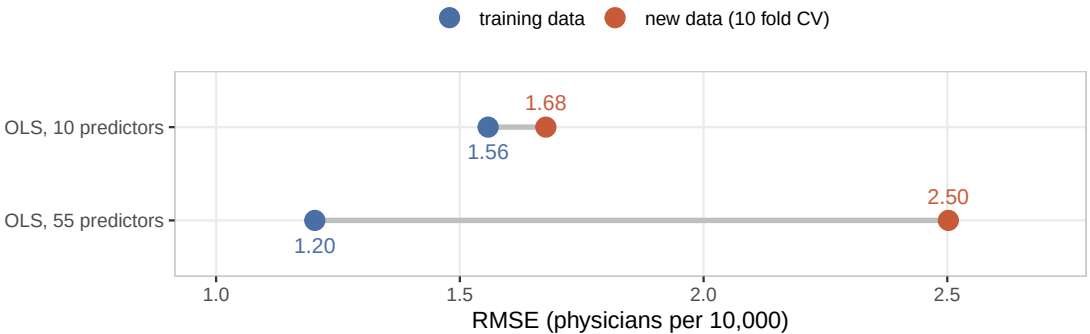
Give it more freedom: every pairwise interaction

```
X55 <- model.matrix(PhysiciansPer10000 ~ .^2, data=counties)[, -1]
ncol(X55)
```

```
[1] 55
```

`.^2` means all main effects plus every pairwise interaction: $10 + 45 = 55$ predictors for 168 counties.

OLS with 55 predictors falls apart



The 55-predictor model looks better on the data it was fit to, and is far worse on new data. Classic overfitting.

Two ways to fight back

When p is large, or close to n, OLS estimates have **high variance**: they chase noise and jump around from sample to sample.

Subset selection	choose which predictors to keep	an on/off switch per variable
Shrinkage	keep all, pull coefficients toward zero	one smooth dial

Last deck was subset selection. Today is shrinkage.

Why not just use subset selection?

- **Best subsets** would need $2^{55} \approx 3.6 \times 10^{16}$ fits. Not happening.
- **Stepwise** is greedy: it can get locked into a path that misses better models.
- Both are unstable: a small change in the data can flip which variables get in.

Shrinkage turns the all-or-nothing choice into a dial we can tune smoothly.

Shrinkage in one sentence

Instead of dropping variables, **shrink** their coefficients toward zero.

- Shrinking **reduces variance**: the estimates stop chasing noise.
- It **adds a little bias**: we accept being slightly wrong on average to be much more stable.
- The trade usually wins on new data. It is the bias-variance tradeoff from week 1.

The idea as a budget

Least squares, with a limit on how big the coefficients can get:

$$\min_{\beta} \sum_{i=1}^{n} \left(Y_i - \beta_0 - \sum_{j=1}^{p} \beta_j X_{ij} \right)^2 \quad \text{subject to} \quad \sum_{j=1}^{p} \beta_j^2 \leq s$$

- s is a **budget**: we may not spend too much on large coefficients.
- A big budget is just OLS. A tiny budget pins every coefficient near zero.

The same idea written as a **penalty**. Choose β to minimize

$$\underbrace{\sum_{i=1}^{n}\left(Y_i - \beta_0 - \sum_{j=1}^{p}\beta_j X_{ij}\right)^2}_{\text{fit (RSS)}} + \underbrace{\lambda\sum_{j=1}^{p}\beta_j^2}_{\text{penalty}}$$

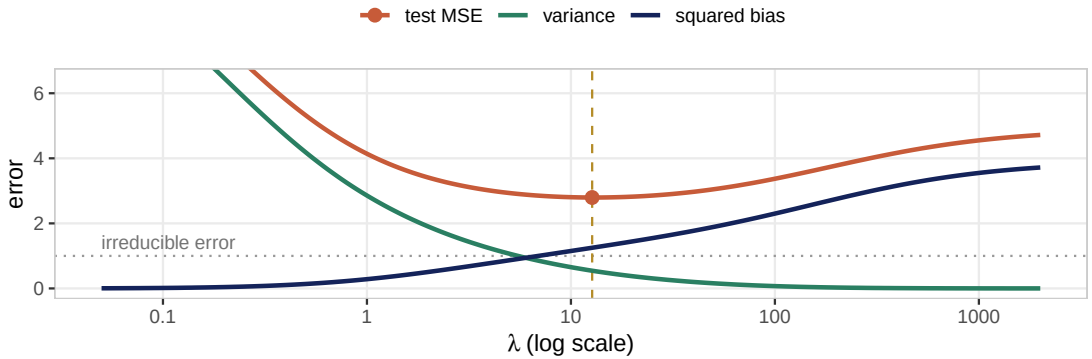
- $\lambda \geq 0$ is the **tuning parameter**: the price of each unit of β_j^2.
- The intercept β_0 is not penalized. Shrinking it would just shift every prediction.

Turning the dial

λ	coefficients	bias and variance
0	ordinary least squares	no bias, highest variance
small	shrunk a little	a little bias, much less variance
large	shrunk a lot	more bias, low variance
$\to \infty$	all $\hat{\beta}_j \to 0$	"ignore every predictor"

The art is picking a λ that does best on new data.

The tradeoff, computed exactly



A simulated problem with $n = 50$ and $p = 45$. Variance falls faster than bias rises, until it doesn't. The gold line is the sweet spot.

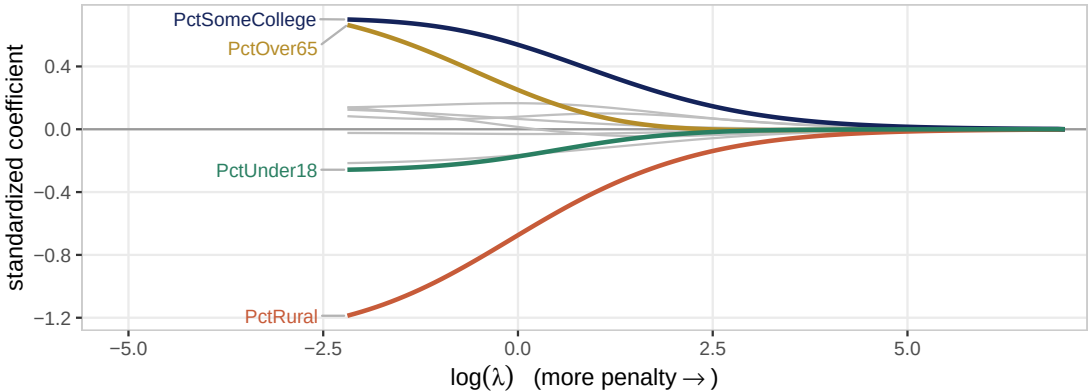
First, put the predictors on the same scale

The penalty $\lambda \sum_j \beta_j^2$ treats every coefficient the same. But:

- `MedianIncome` is in dollars; `PctRural` is in percentage points.
- A "big" coefficient for one might be 0.0001; for the other, 1.5.
- Unscaled, the penalty would hammer some variables purely because of their units.

Center and scale each X_j first. `glmnet` does this for you by default (`standardize=TRUE`).

Ridge on the counties: the coefficient path



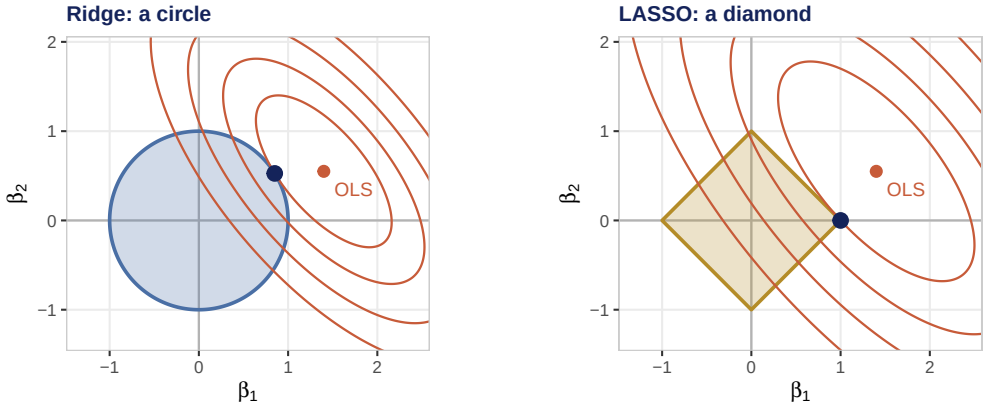
Read right to left: as the penalty loosens, coefficients grow toward OLS. Every one shrinks; none hits exactly zero.

The LASSO: swap squares for absolute values

$$\text{RSS} \; + \; \lambda \sum_{j=1}^{p} |\beta_j|$$

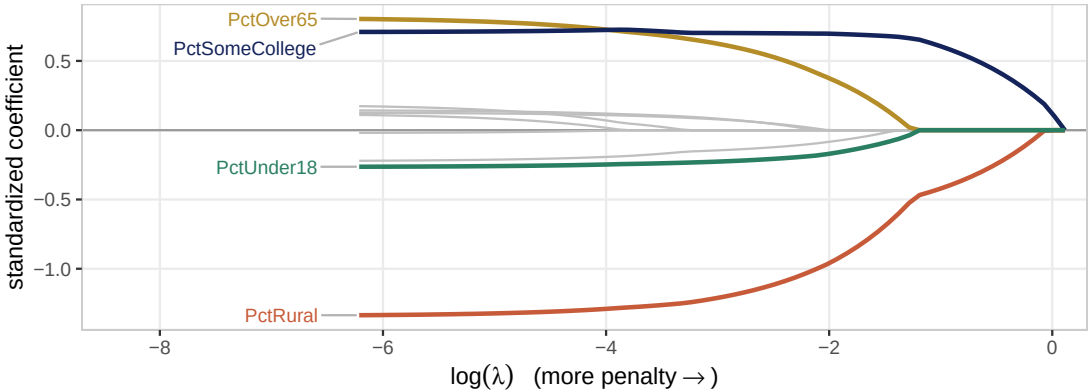
- Ridge uses the **L2** penalty β_j^2. The LASSO uses the **L1** penalty $|\beta_j|$.
- That one change has a big consequence: some $\hat{\beta}_j$ come out exactly zero.
- So the LASSO shrinks *and* selects variables, in one step.

Why the LASSO gives exact zeros



The RSS ellipses grow out from OLS until they touch the budget. The diamond has **corners on the axes**, so the touch point often lands at $\beta_j = 0$ (here $\hat{\beta}_2 = 0$). A circle has no corners.

The LASSO on the counties: the coefficient path



Read right to left: variables enter one at a time, and stay at exactly zero until they do.

How many variables does the LASSO keep?

- At $\lambda = 0$: all 10 predictors, exactly the OLS fit.
- As λ grows, variables typically drop out one by one.
- For large enough λ: none at all, just the intercept.

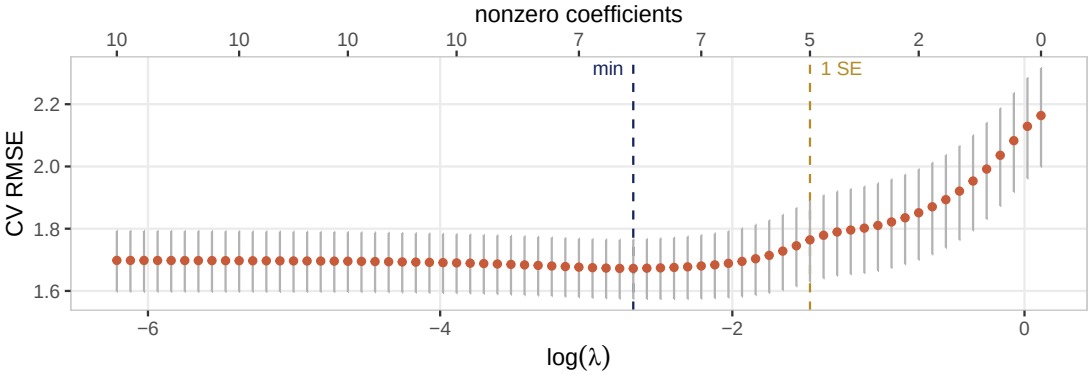
The number of nonzero $\hat{\beta}_j$ is the **effective model size**, and λ sets it.

Choosing λ by cross-validation

1. Lay down a grid of λ values.
2. For each one, compute the K-fold cross-validated error.
3. Pick the λ with the lowest CV error.

`cv.glmnet()` does all three in one call.

The cross-validation curve



LASSO on the 10 county predictors, 10-fold CV, ± 1 standard error. The minimum keeps 7 predictors.

The one-standard-error rule

- The CV curve is **flat** near its minimum, and the error bars overlap.
- So prefer the simplest model whose CV error is within one standard error of the best.
- In `glmnet`: `s = "lambda.1se"` instead of `s = "lambda.min"`.

Here that means 5 predictors instead of 7, with CV error still within one standard error of the best.

Four fits, side by side

	OLS	Ridge	LASSO (min)	LASSO (1 SE)
PctRural	−1.34	−1.19	−1.15	−0.67
PctOver65	0.81	0.66	0.57	0.14
PctSomeCollege	0.71	0.70	0.70	0.68
PctUnder18	−0.27	−0.26	−0.22	−0.08
PctUnemployed	−0.22	−0.22	−0.13	−0.01
PctPoverty	0.19	0.14	·	·
PctUninsured	0.15	0.12	0.06	·
Population	0.13	0.14	0.06	·
MedianIncome	0.12	0.08	·	·
LandArea	−0.02	−0.02	·	·

Standardized coefficients (· = exactly zero). Ridge shrinks everything; the LASSO **zeroes some out**.

Which one should you use?

It depends on the truth, which we never get to see.

- **LASSO** shines when a *few* predictors matter and the rest are noise. Its zeros give a sparse, readable model.
- **Ridge** shines when *many* predictors each matter a little. Shrinking all of them together helps.
- In practice: fit both and let cross-validation decide.

When predictors are correlated

Two near-copies of the same signal, x_1 and x_2, each with a true slope of 1. Three samples:

	OLS		Ridge		LASSO	
	$\hat{\beta}_1$	$\hat{\beta}_2$	$\hat{\beta}_1$	$\hat{\beta}_2$	$\hat{\beta}_1$	$\hat{\beta}_2$
Sample A	2.12	0.03	0.91	0.87	1.78	0.00
Sample B	−0.01	2.01	0.81	0.87	0.00	1.70
Sample C	1.13	0.96	0.90	0.89	1.07	0.72

OLS lurches from sample to sample. Ridge **shares** the weight every time. The LASSO often drops one twin, and which one is arbitrary.

Elastic net: a blend of both

$$\text{RSS} \;+\; \lambda \left[(1-\alpha) \sum_{j=1}^{p} \beta_j^2 \;+\; \alpha \sum_{j=1}^{p} |\beta_j| \right]$$

$\alpha = 0$	pure ridge
$\alpha = 1$	pure LASSO
$\alpha = 0.5$	an even mix: some zeros, plus grouped shrinkage of correlated predictors

`glmnet` actually minimizes $\frac{1}{2n}\text{RSS} + \lambda\left[\frac{1-\alpha}{2}\sum \beta_j^2 + \alpha \sum |\beta_j|\right]$, so its λ is on a different scale than ours. Same idea. `alpha=0.5` is a sensible first try.

The whole workflow in glmnet

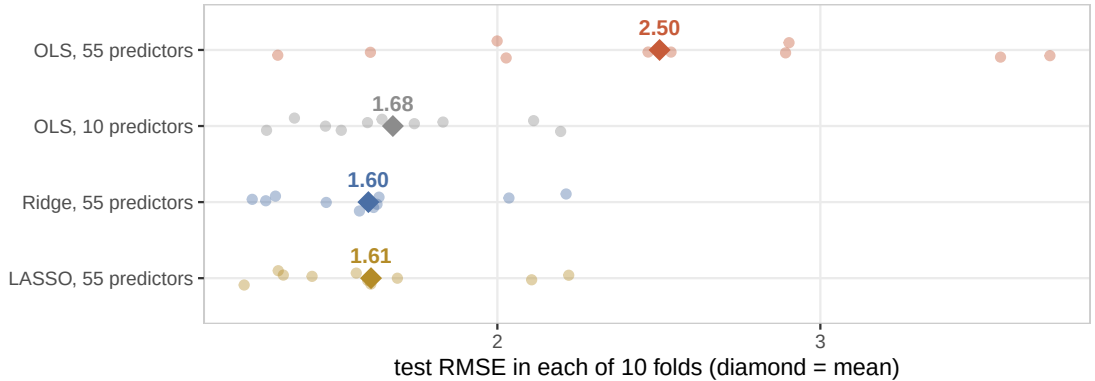
```r
library(glmnet)
X <- model.matrix(PhysiciansPer10000 ~ ., data=counties)[, -1]   # predictors as a matrix
y <- counties$PhysiciansPer10000

fit.ridge <- glmnet(X, y, alpha=0)          # alpha = 0: ridge
fit.lasso <- glmnet(X, y, alpha=1)          # alpha = 1: LASSO (0.5: elastic net)
plot(fit.lasso, xvar="lambda")              # the coefficient path

cv.lasso <- cv.glmnet(X, y, alpha=1)        # 10-fold CV over a grid of lambdas
cv.lasso$lambda.min                         # lambda with the lowest CV error
coef(cv.lasso, s="lambda.1se")              # coefficients under the 1-SE rule
predict(cv.lasso, newx=X[1:3, ], s="lambda.min")
```

glmnet wants a numeric matrix, not a formula; model.matrix builds it, dummies and all.

Back to the 55 predictors



Same 55 predictors that wrecked OLS. With a penalty chosen by CV, ridge and LASSO beat even the small model.

What to take away

- **Ridge** and the **LASSO** trade a little bias for a lot less variance by shrinking coefficients toward zero.
- Ridge shrinks every coefficient; the LASSO sets some exactly to zero, so it also selects variables.
- λ is the dial. Choose it by **cross-validation**, and consider the 1-SE rule for a simpler model.
- Standardize first (`glmnet` does). Correlated predictors? Try the **elastic net**.